



THIẾT KẾ HỆ THỐNG SỐ VỚI HDL

BÁO CÁO ĐỒ ÁN

THIẾT KẾ BỘ TĂNG TỐC XỬ LÝ NHÂN MA TRẬN THUẦN TÚY TRONG MÔ HÌNH QWEN2 ĐÃ LƯỢNG TỬ HÓA

- **Vũ Đại Dương** 23520359
- **Nguyễn Đặng Phương Duy** 23520372



Nội dung báo cáo

- 1 Tổng quan đề tài
- 2 Kiến trúc hệ thống
- 3 Thiết kế hệ thống
- 4 Thực nghiệm hệ thống trên FPGA
- 5 Kết luận

01

Tổng quan đề tài





Bối cảnh

- Các mô hình ngôn ngữ lớn (LLM) như Qwen2 mang trong mình kiến trúc Transformer đòi hỏi khối lượng tính toán khổng lồ.
 - Các phép nhân ma trận thuần túy chiếm tỷ trọng cao về cả thời gian xử lý lẫn năng lượng tiêu thụ
- => Đề tài tập trung thiết kế một bộ tăng tốc nhân ma trận trên FPGA để giải quyết nút thắt cho mô hình Qwen2



===== BÁO CÁO PHÂN TÍCH TỶ TRỌNG (STRESS TEST 55s) =====
 ⌚ Tổng thời gian chạy toàn bộ mô hình: 2.2645 giây

1. Attention - Q Proj (q_proj)	: 0.2638 s	Chiếm: 11.65%
2. Attention - K Proj (k_proj)	: 0.0513 s	Chiếm: 2.26%
3. Attention - V Proj (v_proj)	: 0.0097 s	Chiếm: 0.43%
4. Attention - O Proj (o_proj)	: 0.0017 s	Chiếm: 0.07%
5. Mạng FFN (gate, up, down_proj, fc1, fc2)	: 0.0778 s	Chiếm: 3.44%
6. Audio Encoder (Conv1d, Conv2d)	: 0.7626 s	Chiếm: 33.68%
7. Attention Core (QxK, Softmax, SV)	: 0.3252 s	Chiếm: 14.36%
8. Các lớp Linear phân loại khác	: 0.0158 s	Chiếm: 0.70%

TỔNG TỶ TRỌNG CÁC LỚP MA TRẬN THUẦN TÚY (Thuần MatMul): 18.55%
 (Thời gian thực thi: 0.4200 giây)



Một phần kiến trúc của Qwen2



Nội dung thực hiện

Quá trình thực hiện bao gồm các bước cốt lõi:

- Trích xuất và đánh giá: Trích xuất ma trận thực tế từ mô hình Qwen2 và phân tích tỷ trọng thời gian thực thi.
- Lượng tử hóa: Áp dụng kỹ thuật lượng tử hóa đối xứng để chuyển đổi dữ liệu từ định dạng dấu phẩy động 32-bit (FP32) xuống số nguyên 8-bit (INT8)
- Thiết kế phần cứng: Thiết kế kiến trúc Systolic Array 16x16 với luồng dữ liệu Output Stationary. Dữ liệu được truyền tải bằng giao thức AXI-DMA kết hợp bus AXI4-Stream vào bộ đệm BRAM Ping-Pong.
- Xử lý hậu kỳ phần cứng: Xây dựng module cộng dồn các khối Tiling được lưu trong bộ nhớ FWFT BRAM, sau đó Rescale kết quả từ INT32 về lại INT8.
- Kiểm chứng: Sử dụng Python để so sánh độ chính xác và thời gian (timing) giữa kết quả xuất ra từ hệ thống FPGA và kết quả tính toán bằng thư viện phần mềm.



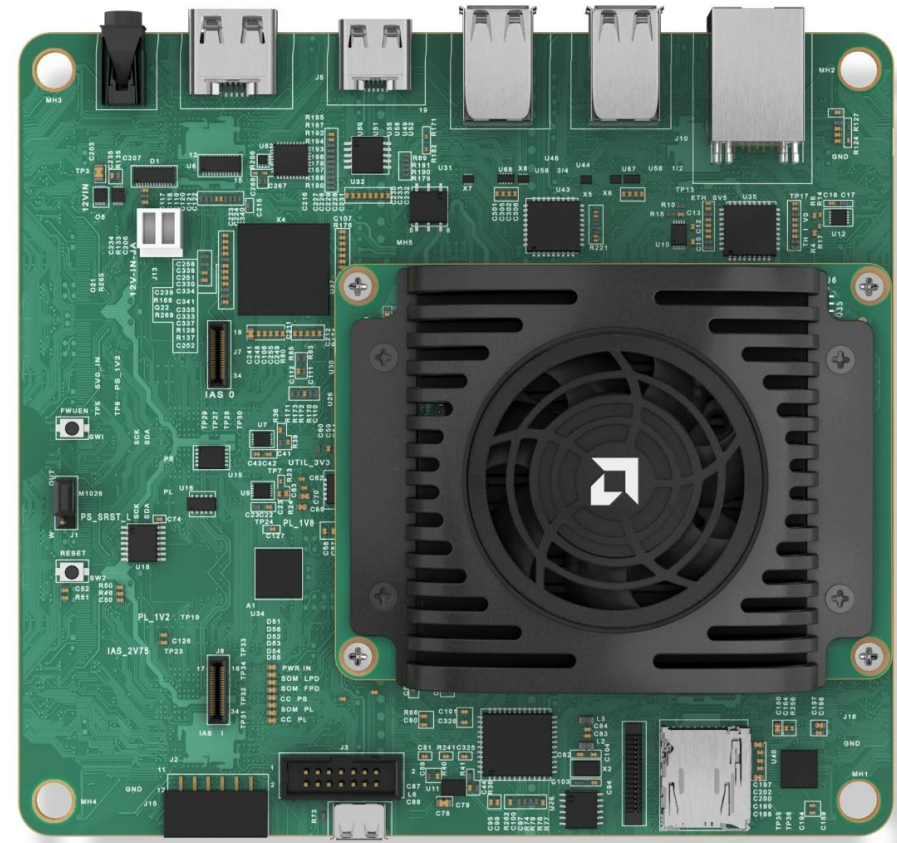
Kit AMD Kria KV260

Phần FPGA (Programmable Logic - PL)

- System Logic Cells: 256K
- Block RAM Blocks: 144 (tương đương khoảng 5.1Mb)
- UltraRAM Blocks: 64
- DSP Slices: 1248
- Tổng bộ nhớ On-Chip (SRAM): 26.6 Mb (tổng hợp từ BRAM, URAM và Distributed RAM).

Phần Vi xử lý (Processing System - PS)

- CPU chính: Quad-core (4 nhân) Arm® Cortex®-A53 hoạt động ở xung nhịp lên đến 1.33 GHz - 1.5 GHz.
- DRAM: 4 GB DDR4



02

Kiến trúc hệ thống



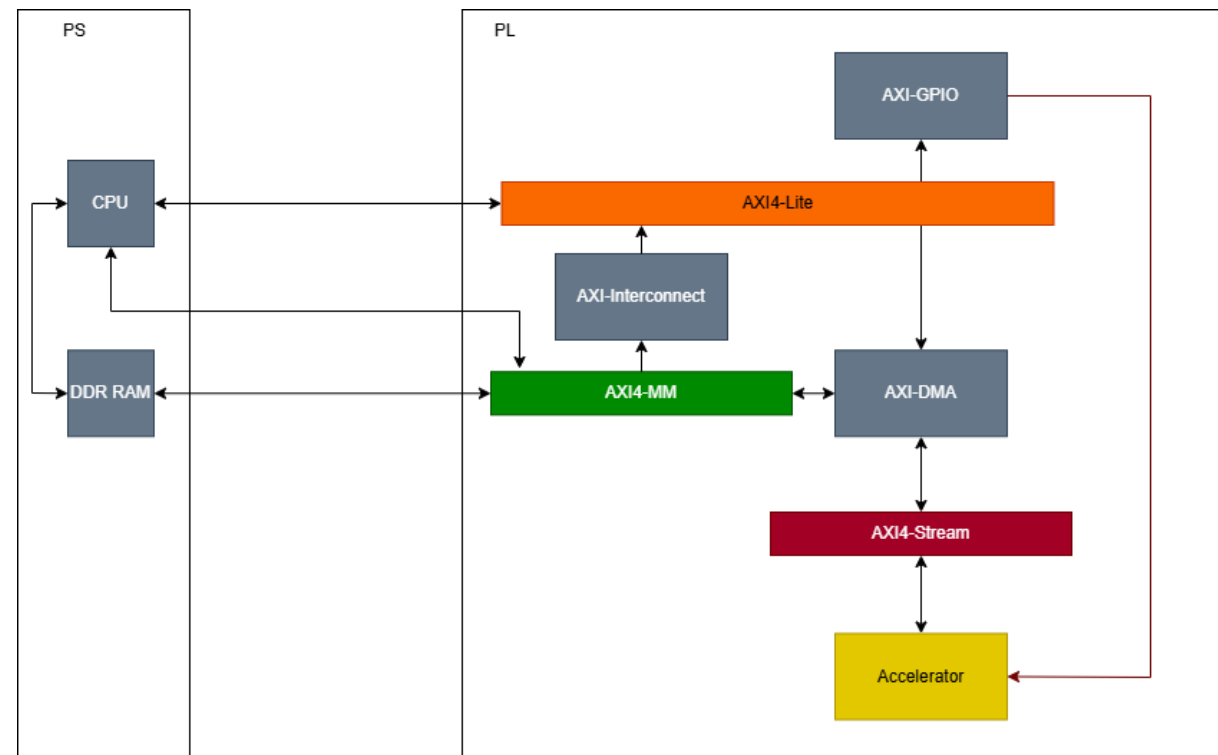


Kiến trúc tổng quan

Kết hợp chặt chẽ giữa hai thành phần: Hệ thống xử lý (Processing System - PS) và Hệ thống logic lập trình được (Programmable Logic - PL)

Sử dụng hệ thống bus AXI bao gồm:

- AXI4-Lite: được CPU sử dụng để truyền các lệnh cấu hình và kiểm tra trạng thái của các thanh ghi điều khiển bên trong các khối IP.
- AXI4 Memory-Mapped (AXI4-MM): sử dụng để đọc từ bộ nhớ DDRAM trong PS với AXI-DMA trong PL
- AXI4-Stream: sử dụng để truyền dữ liệu tốc độ cao từ AXI-DMA tới bộ tăng tốc



Sơ đồ kiến trúc tổng quan

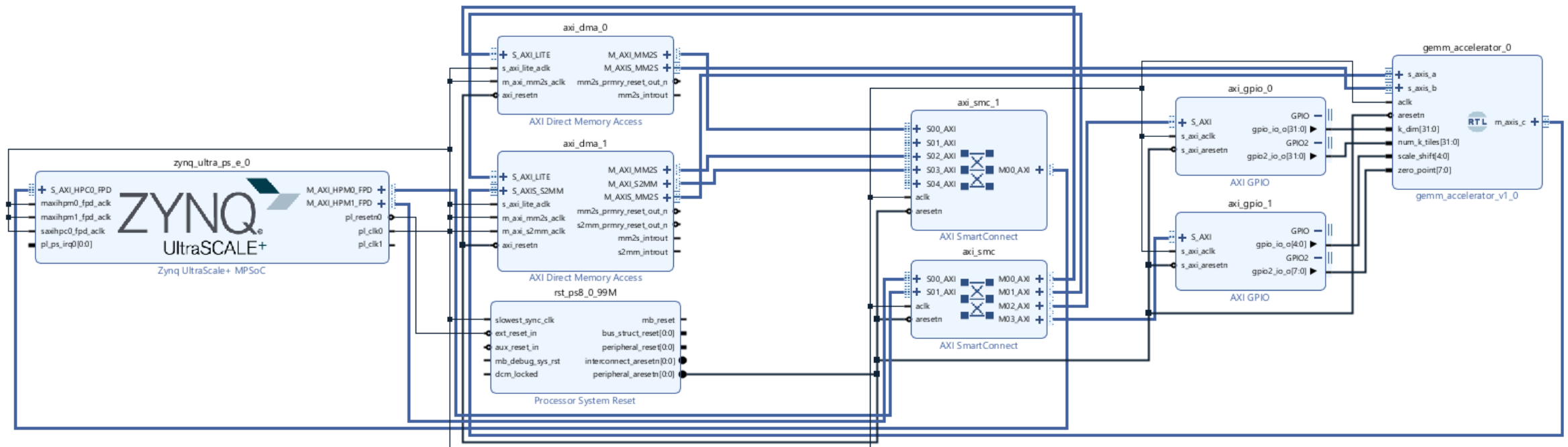
03

Thiết kế hệ thống





Chi tiết hệ thống



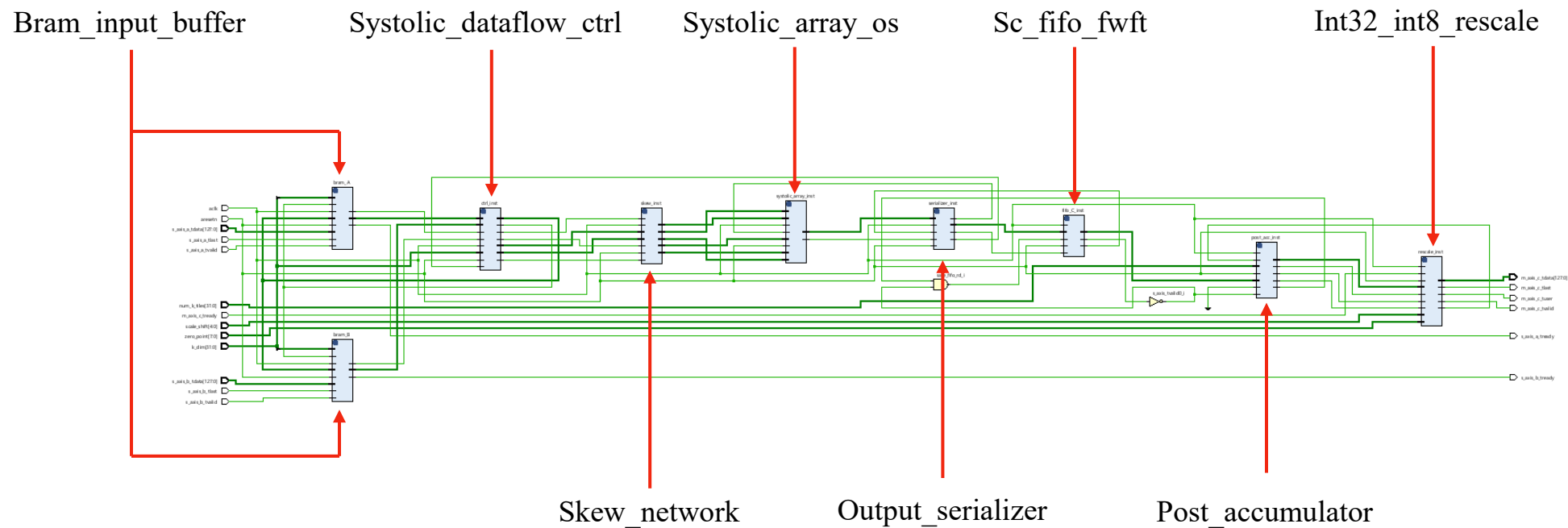


Cây thư mục

- ▼ Design Sources (2)
 - ▼  **gemm_block_wrapper** (gemm_block_wrapper.v) (1)
 - ▼  **gemm_block_i** : gemm_block (gemm_block.bd) (1)
 - ▼  **gemm_block** (gemm_block.v) (9)
 - >  axi_dma_0 : gemm_block_axi_dma_0_0 (gemm_block_axi_dma_0_0.xci)
 - >  axi_dma_1 : gemm_block_axi_dma_1_0 (gemm_block_axi_dma_1_0.xci)
 - >  axi_gpio_0 : gemm_block_axi_gpio_0_0 (gemm_block_axi_gpio_0_0.xci)
 - >  axi_gpio_1 : gemm_block_axi_gpio_1_0 (gemm_block_axi_gpio_1_0.xci)
 - >  axi_smc : gemm_block_axi_smc_0 (gemm_block_axi_smc_0.xci)
 - >  axi_smc_1 : gemm_block_axi_smc_1_0 (gemm_block_axi_smc_1_0.xci)
 - ▼  **gemm_accelerator_0** : gemm_block_gemm_accelerator_0_2 (Module Reference Wrapper) (1)
 - ▼  **gemm_block_gemm_accelerator_0_2** (gemm_block_gemm_accelerator_0_2.v) (1)
 - ▼  **inst** : gemm_accelerator (gemm_accelerator.v) (9)
 -  bram_A : bram_input_buffer (bram_input_buffer.v)
 -  bram_B : bram_input_buffer (bram_input_buffer.v)
 -  ctrl_inst : systolic_dataflow_ctrl (systolic_dataflow_ctrl.v)
 - >  skew_inst : skew_network (skew_network.v) (80)
 - >  systolic_array_inst : systolic_array_os (systolic_array_os.v) (256)
 -  serializer_inst : output_serializer (output_serializer.v)
 -  fifo_C_inst : sc_fifo_fwft (sc_fifo_fwft.v)
 -  post_acc_inst : post_accumulator (post_accumulator.v)
 -  rescale_inst : int32_int8_rescale (int32_int8_rescale.v)
 - >  rst_ps8_0_99M : gemm_block_rst_ps8_0_99M_0 (gemm_block_rst_ps8_0_99M_0.xci)
 - >  zynq_ultra_ps_e_0 : gemm_block_zynq_ultra_ps_e_0_0 (gemm_block_zynq_ultra_ps_e_0_0.xci)



Khối gemm_accelerator





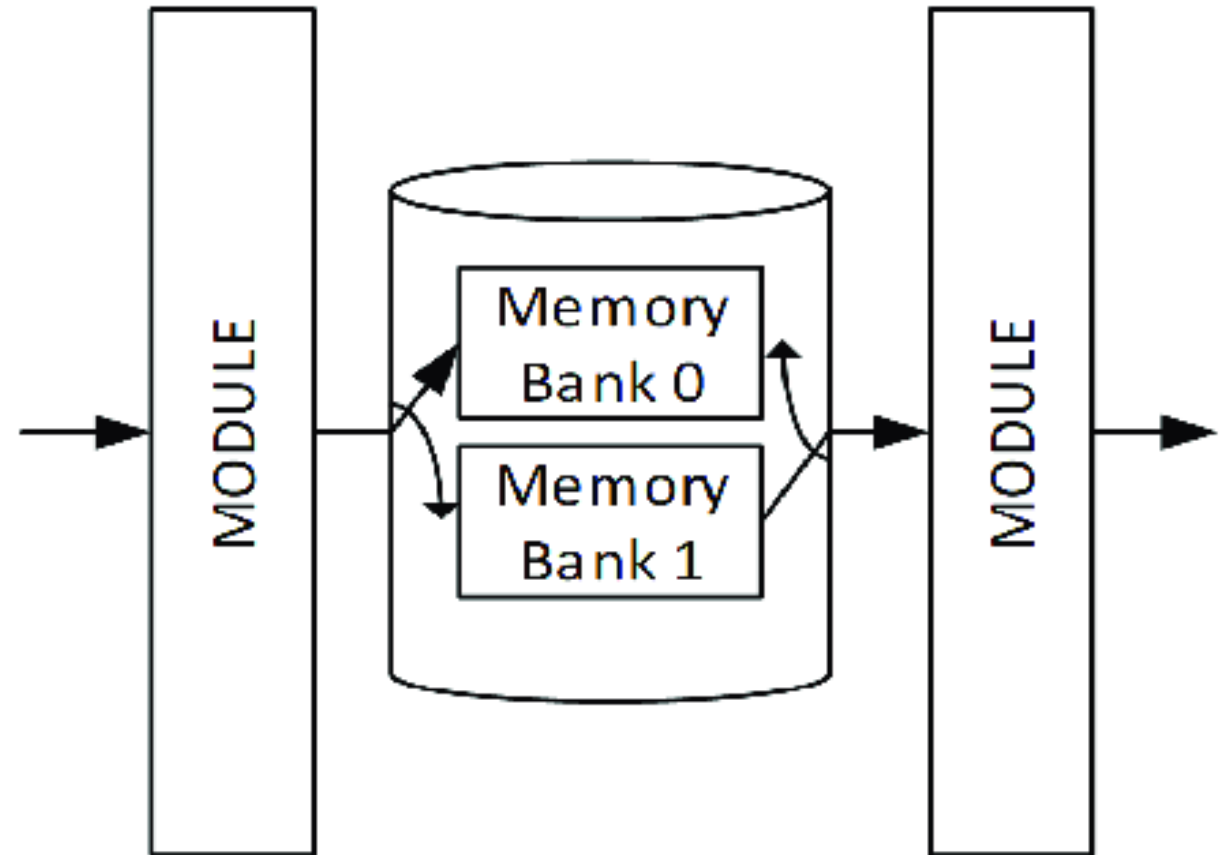
Module bram_input_buffer

Sử dụng kỹ thuật Ping-Pong Buffer thay phiên nhau đọc dữ liệu từ AXI-DMA để cung cấp cho lõi tính toán

Trong quá trình hoạt động:

- Con trỏ ghi: Tiếp nhận dữ liệu liên tục từ DMA và ghi vào một phân vùng.
- Con trỏ đọc: Khi điều khiển SA sẽ hút dữ liệu ra từ phân vùng còn lại để đưa vào tính toán.

=> Cho phép luồng dữ liệu tính toán được diễn ra liên tục, giảm độ trễ truy xuất bộ nhớ DDR

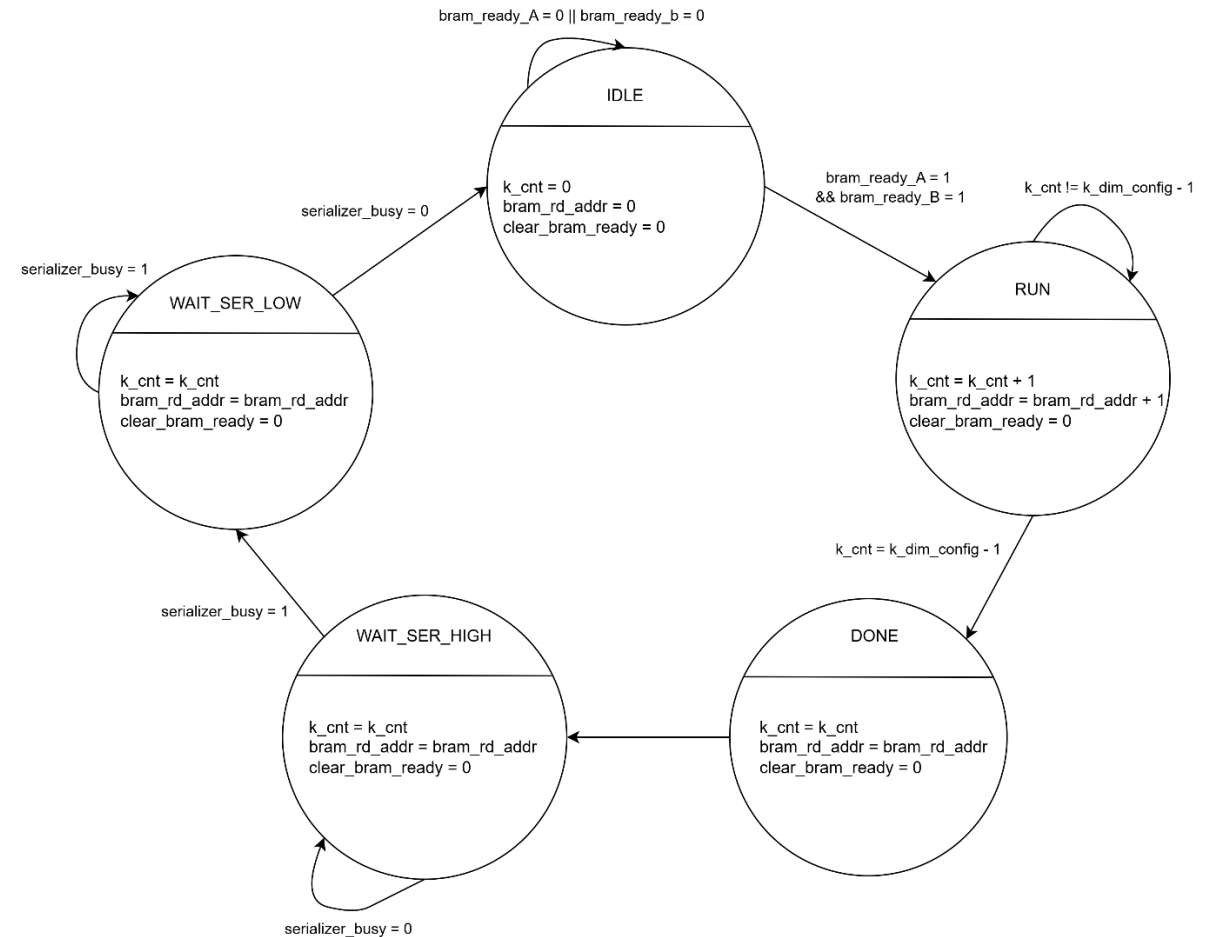


Sơ đồ tổng quan Ping-Pong Buffer



Module Systolic_dataflow_ctrl

Là một máy trạng thái hữu hạn (Finite State Machine – FSM) chịu trách nhiệm tạo ra địa chỉ đọc tuần tự để quét qua bộ đệm BRAM dọc theo chiều K của khối ma trận. Nó đồng thời sinh ra các tín hiệu điều khiển quan trọng như cờ hợp lệ (valid), cờ xóa bộ cộng dồn (clear_acc) và cờ kết thúc khối (last_mac).



FSM của module systolic_dataflow_ctrl

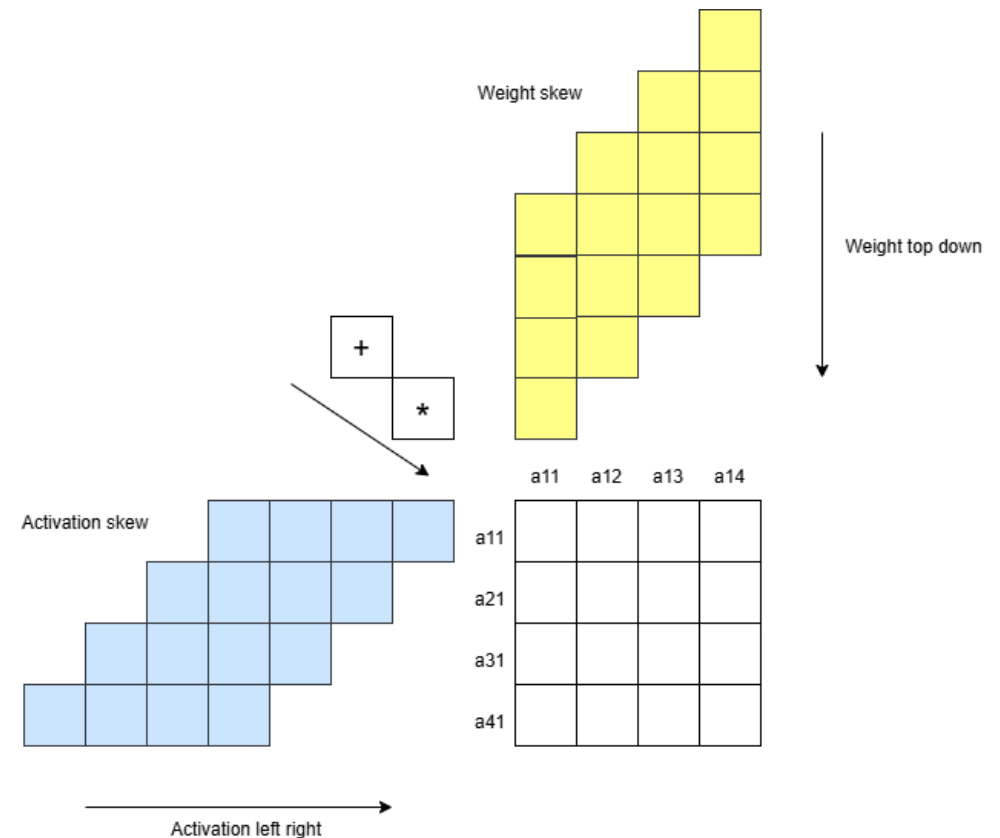


Module

Skew_network

Chức năng chính: Tạo độ trễ bậc thang cho dữ liệu đầu vào (weight, act) và các tín hiệu điều khiển tương ứng trước khi đưa vào mảng Systolic array

=> Giúp các cặp dữ liệu tương ứng gặp nhau chính xác tại cùng một phần tử xử lý ở một thời điểm



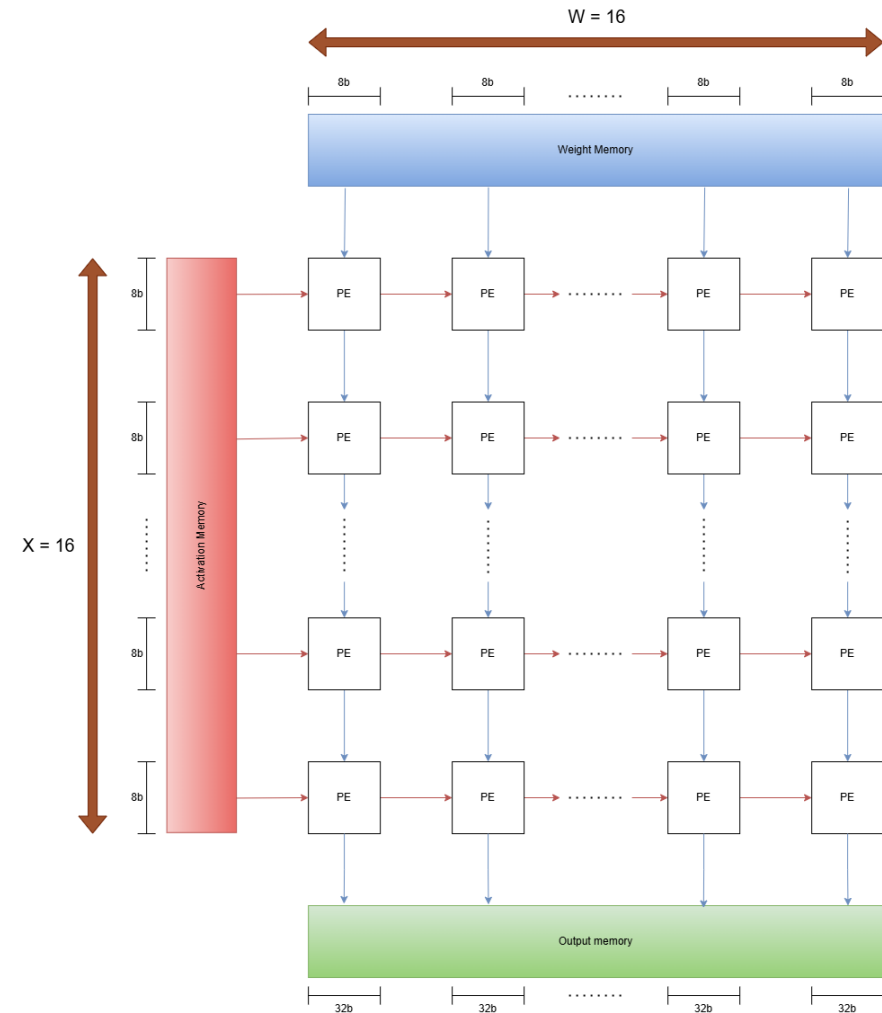
Minh họa skew network cho mảng 4x4



Module Systolic_array_os

Module `mac_core` thực hiện phép nhân hai toán hạng nguyên 8-bit và cộng dồn vào thanh ghi chứa 32-bit, được thiết kế cực kỳ tối ưu bằng cách phân bổ các thanh ghi nội bộ của các khối xử lý tín hiệu số DSP48E2

Tại mỗi module `pe_wrapper`, kết quả cộng dồn được giữ nguyên tại chỗ trong suốt quá trình quét dọc theo chiều K. Ma trận Act trôi ngang từ trái sang phải, ma trận Weight trôi dọc từ trên xuống dưới. Khi tín hiệu `last_mac` xuất hiện, toàn bộ 16 hàng PE sẽ đồng loạt xả dữ liệu kết quả xuống hàng PE dưới cùng.



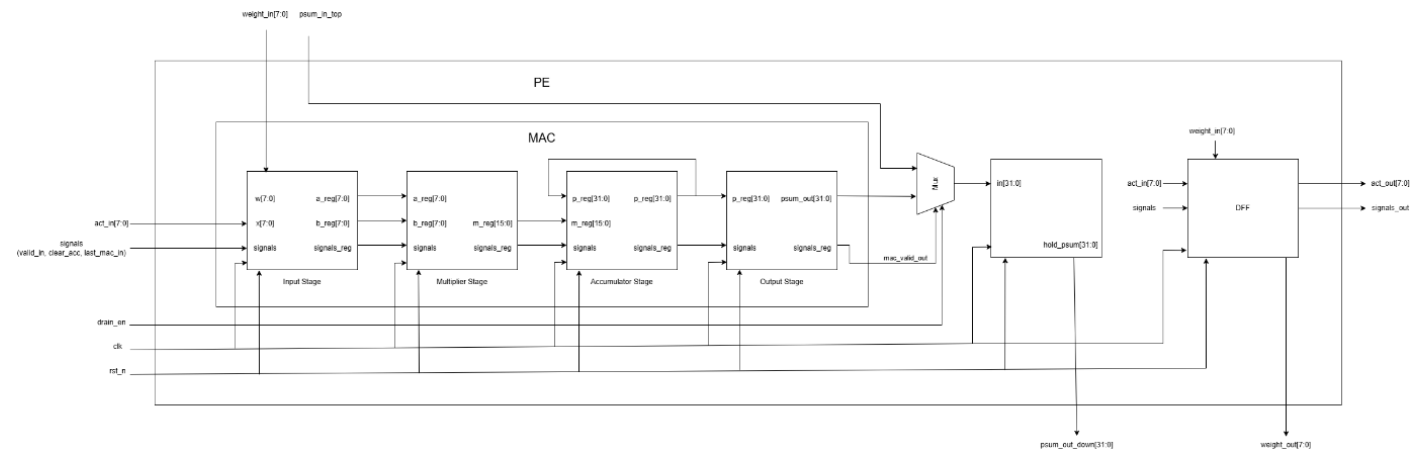
Module Mac_core



Module Systolic_array_os

Module `mac_core` thực hiện phép nhân hai toán hạng nguyên 8-bit và cộng dồn vào thanh ghi chứa 32-bit, được thiết kế cực kỳ tối ưu bằng cách phân bổ các thanh ghi nội bộ của các khối xử lý tín hiệu số DSP48E2

Tại mỗi module `pe_wrapper`, kết quả cộng dồn được giữ nguyên tại chỗ trong suốt quá trình quét dọc theo chiều K. Ma trận Act trôi ngang từ trái sang phải, ma trận Weight trôi dọc từ trên xuống dưới. Khi tín hiệu `last_mac` xuất hiện, toàn bộ 16 hàng PE sẽ đồng loạt xả dữ liệu kết quả xuống hàng PE dưới cùng.



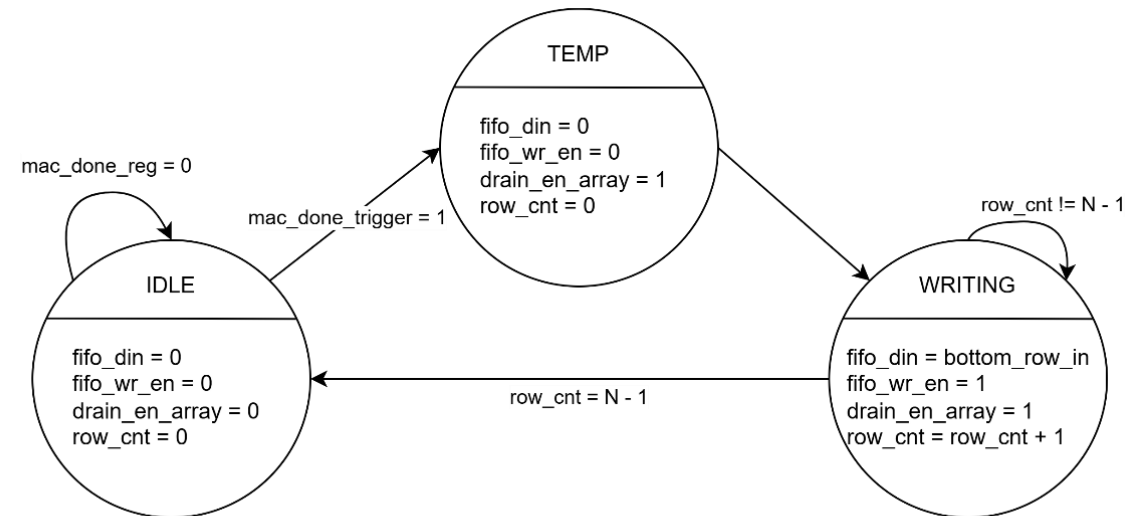
Module PE_wrapper



Module Output_serializer

Module output_serializer chụp lấy dải dữ liệu song song rộng 512-bit từ hàng dưới cùng của Systolic Array, tương ứng với 16 phần tử kết quả INT32 được xuất đồng thời.

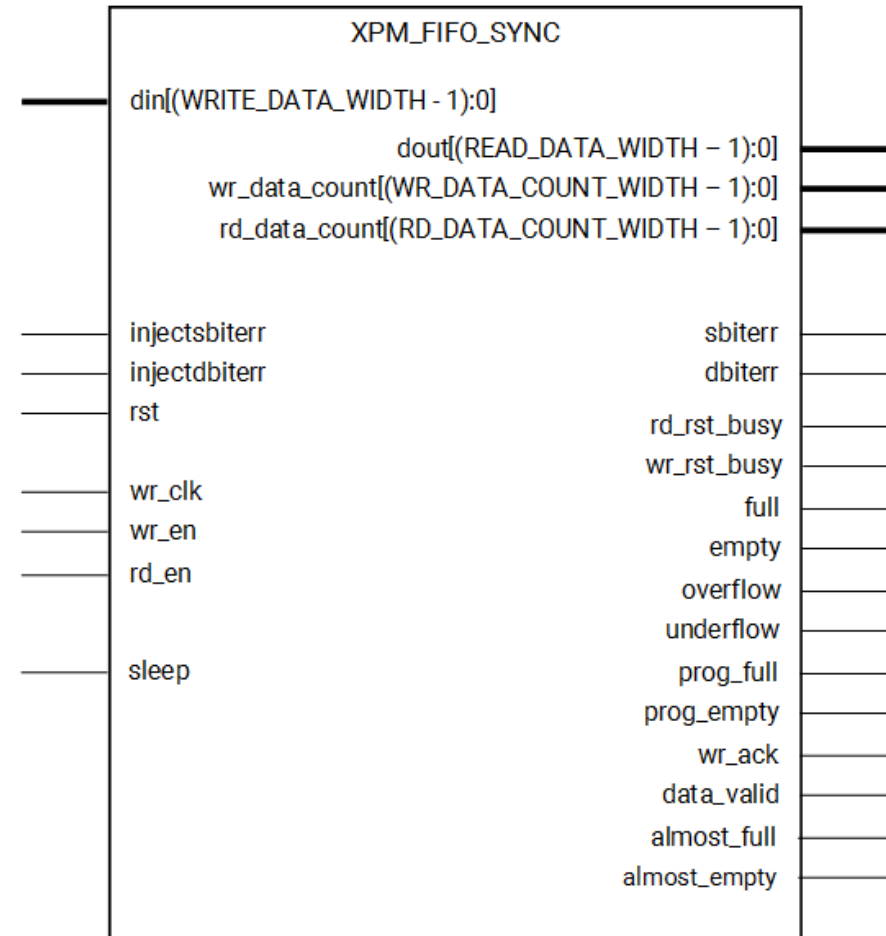
Do bus AXI4-Stream phía sau chỉ xử lý dữ liệu theo dạng tuần tự, module này thực hiện tách tuần tự từng phần tử INT32 và đẩy vào bộ đệm FIFO.





Module SC_FIFO_FWFT

FIFO được xây dựng bằng Hard IP xpm_fifo_sync ở chế độ FWFT, cho phép dữ liệu đầu tiên xuất hiện ngay tại ngõ ra mà không cần thêm chu kỳ đọc phụ, từ đó giảm độ trễ toàn hệ thống.



X17929-061419



Module post_accumulator

Mỗi lần quét chỉ tạo ra một kết quả tích lũy tạm thời:

$$C = \sum_{t=0}^{T-1} X_i \cdot W_i$$

Module post_accumulator duy trì một vùng RAM nội bộ để lưu các tổng trung gian. Sau mỗi lần SA hoàn tất một tile, kết quả INT32 mới sẽ được cộng dồn với giá trị trước đó:

$$C_{acc}^{t+1} = C_{acc}^t + C_t$$

Khi bộ đếm đạt đủ số lượng tile được cấu hình bởi phần mềm, tổng INT32 cuối cùng mới được xuất sang khối tái chuẩn hóa.

Module int32_int8_rescale

Module int32_int8_rescale thực hiện phép lượng tử hóa đối xứng bằng phần cứng theo công thức:

$$q = x \gg \{shift_val\} + z_p.$$

Sau bước scale và cộng zero_point, kết quả tiếp tục đi qua mạch xén biên:

$$q_{final} = \begin{cases} 127 & q > 127 \\ -128 & q < -128 \\ q & otherwise \end{cases}$$

Khối này đảm bảo dữ liệu cuối cùng luôn nằm trong miền biểu diễn hợp lệ của INT8.



04

Thực nghiệm hệ thống trên FPGA



Tài nguyên phần cứng sử dụng

Tài nguyên	Sử dụng	Tài nguyên trên FPGA	% sử dụng
LUT	28622	117120	22.44
LUTRAM	11171	57600	19.39
FF	35690	234240	15.24
BRAM	34	144	23.61
DSP	256	1248	20.51

Name	CLB LUTs (117120)	CLB Registers (234240)	CARRY8 (14640)	F7 Muxes (58560)	CLB (14640)	LUT as Logic (117120)	LUT as Memory (57600)	Block RAM Tile (144)	DSPs (1248)	BUFG_PS (96)	PS8 (1)
gemm_block_wrapper	28622	35690	358	1026	6303	17451	11171	34	256	1	1
gemm_block_i (gemm_block)	28622	35690	358	1026	6303	17451	11171	34	256	1	1
axi_dma_0 (gemm_block_axi)	693	984	9	0	197	641	52	8.5	0	0	0
axi_dma_1 (gemm_block_axi)	2014	3066	23	0	516	1879	135	11	0	0	0
axi_gpio_0 (gemm_block_axi)	63	222	0	0	32	63	0	0	0	0	0
axi_gpio_1 (gemm_block_axi)	43	74	0	0	17	43	0	0	0	0	0
axi_smc (gemm_block_axi)	1812	1788	10	2	363	1805	7	0	0	0	0
axi_smc_1 (gemm_block_axi)	2668	4806	0	0	621	1983	685	0	0	0	0
gemm_accelerator_0 (gemm_block)	21321	24717	316	1024	4803	11030	10291	14.5	256	0	0
inst (gemm_block_gemm)	21321	24717	316	1024	4803	11030	10291	14.5	256	0	0
bram_A (gemm_block)	5948	61	2	512	901	1212	4736	0	0	0	0
bram_B (gemm_block)	5938	57	2	512	927	1202	4736	0	0	0	0
ctrl_inst (gemm_block)	163	103	14	0	74	163	0	0	0	0	0
fifo_C_inst (gemm_block)	55	51	64	0	83	55	0	14.5	0	0	0
post_acc_inst (gemm_block)	3904	572	234	0	716	3386	518	0	0	0	0
rescale_inst (gemm_block)	18	130	0	0	48	18	0	0	0	0	0
serializer_inst (gemm_block)	11	524	0	0	149	11	0	0	0	0	0
skew_inst (gemm_block)	297	0	0	0	72	12	285	0	0	0	0
systolic_array_inst (gemm_block)	5014	23219	0	0	2794	4998	16	0	256	0	0



Tần số sau khi chạy implementation

Kết quả báo cáo timing sau khi chạy impl có được bảng bên

Từ đó có thể tính được tần số F_{max} của mạch sẽ là:

$$F_{max} = \frac{1}{\frac{1}{F_{clock}} - WNS} \approx 129MHz$$

Thông số	Giá trị
Worst Negative Slack (WNS)	2.238ns
Total Negative Slack (TNS)	0 ns
Worst Hold Slack (WHS)	0.010 ns
Total Hold Slack (THS)	0 ns
Clock Frequency	100 MHz



Tài nguyên phần cứng sử dụng

Phần lớn tài nguyên thiết kế được sử dụng tại khối `gemm_accelerator_0` trong đó:

- Module `systolic_array_inst` sử dụng lượng lớn DSP (256) và CLB Registers (23219)
- Module `bram_A` và `bram_B` sử dụng lượng lớn LUT as memory (4736x2)
- Module `post_acc_inst` sử dụng lượng lớn LUT as logic (3386) và CLB LUTs (3904)

Name	CLB LUTs (117120)	CLB Registers (234240)	CARRY8 (14640)	F7 Muxes (58560)	CLB (14640)	LUT as Logic (117120)	LUT as Memory (57600)	Block RAM Tile (144)	DSPs (1248)	BUFG_PS (96)	PS8 (1)
▼ gemm_block_wrapper	28622	35690	358	1026	6303	17451	11171	34	256	1	1
▼ gemm_block_i (gemm_block)	28622	35690	358	1026	6303	17451	11171	34	256	1	1
> axi_dma_0 (gemm_block_axi)	693	984	9	0	197	641	52	8.5	0	0	0
> axi_dma_1 (gemm_block_axi)	2014	3066	23	0	516	1879	135	11	0	0	0
> axi_gpio_0 (gemm_block_axi)	63	222	0	0	32	63	0	0	0	0	0
> axi_gpio_1 (gemm_block_axi)	43	74	0	0	17	43	0	0	0	0	0
> axi_smc (gemm_block_axi)	1812	1788	10	2	363	1805	7	0	0	0	0
> axi_smc_1 (gemm_block_axi)	2668	4806	0	0	621	1983	685	0	0	0	0
▼ gemm_accelerator_0 (gemm_block)	21321	24717	316	1024	4803	11030	10291	14.5	256	0	0
▼ inst (gemm_block_gemm)	21321	24717	316	1024	4803	11030	10291	14.5	256	0	0
bram_A (gemm_block)	5948	61	2	512	901	1212	4736	0	0	0	0
bram_B (gemm_block)	5938	57	2	512	927	1202	4736	0	0	0	0
ctrl_inst (gemm_block)	163	103	14	0	74	163	0	0	0	0	0
> fifo_C_inst (gemm_block)	55	51	64	0	83	55	0	14.5	0	0	0
post_acc_inst (gemm_block)	3904	572	234	0	716	3386	518	0	0	0	0
rescale_inst (gemm_block)	18	130	0	0	48	18	0	0	0	0	0
serializer_inst (gemm_block)	11	524	0	0	149	11	0	0	0	0	0
skew_inst (gemm_block)	297	0	0	0	72	12	285	0	0	0	0
> systolic_array_inst (gemm_block)	5014	23219	0	0	2794	4998	16	0	256	0	0



Công suất tiêu thụ

Phần lớn công suất nằm ở công suất động (Dynamic) trong đó đến từ hoạt động chuyển mạch của logic số và các khối DSP trong quá trình tính toán

Phần PS chiếm 90% công suất động do phải chạy các thành phần như CPU, DDR, hệ thống bus và các ngoại vi trong kit

Phần PL chỉ tiêu thụ lượng nhỏ công suất

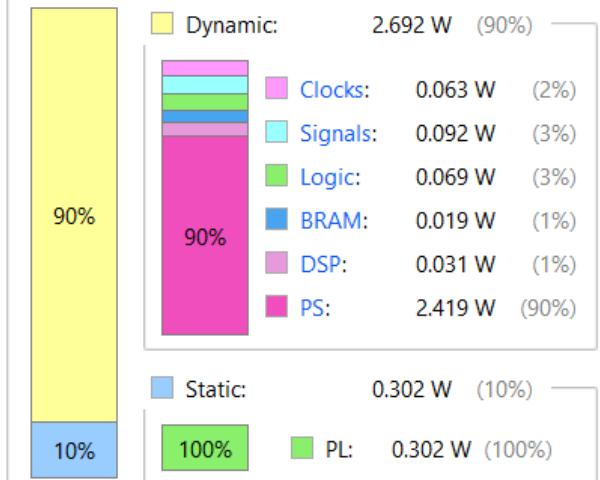
Nhiệt độ của hệ thống nằm ở mức tiêu chuẩn khi hoạt động ở tần số 100MHz

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	2.995 W
Design Power Budget:	Not Specified
Process:	typical
Power Budget Margin:	N/A
Junction Temperature:	31.9°C
Thermal Margin:	53.1°C (22.6 W)
Ambient Temperature:	25.0 °C
Effective θ_{JA} :	2.3°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power





So sánh kết quả test bench và kit



```

debian@KV260-01:~/pduy/test_new_ver$
debian@KV260-01:~/pduy/test_new_ver$ gcc 16x512.c -o r16
debian@KV260-01:~/pduy/test_new_ver$ sudo ./r16

>> Pure NPU Compute Time : 21710 ns (21.710 us)

```

Thời gian testbench: 14.28us với $T = 10\text{ns}$

Thời gian nạp KIT: 21.71us



Kết quả chạy trực tiếp trên kit KV260

```
[1/4] Loading INT8 data from binary files... Done. Loaded 524288 bytes successfully.
[2/4] Running CPU Baseline Matrix Multiplication...
[CPU Baseline Progress] [=====] 100%
CPU Baseline Done in 4730.597 ms.

[3/4] Running NPU: 512x512 matmul | K_DIM=32 | TILES=16 | SHIFT=10

=====
>> PERFORMANCE & ACCURACY COMPARISON REPORT <<
=====
Matrix Size           : 512 x 512 x 512 (INT8)
1. CPU Execution Time   : 4730.597 ms
2. Pure NPU Hardware Time : 47.381 ms (DMA TX + RX Wait)
3. Total NPU System Time : 242.882 ms (Includes Linux overhead)
=====
>> SPEEDUP (Pure Hardware) : 99.84x FASTER than CPU
>> SPEEDUP (Total System)  : 19.48x FASTER than CPU
=====
>> SYSTEM LEVEL TIMING PROFILING REPORT <<
=====
1. Software Prep Time (T_sw_prep) : 190.622 ms ( 79.4%)
2. DMA Transmit + Core (T_dma_tx)  : 45.733 ms ( 19.0%)
3. DMA Receive Time (T_dma_rx)     : 1.648 ms ( 0.7%)
4. Software Post Time (T_sw_post)   : 2.154 ms ( 0.9%)
=====
>> Hardware Compute Efficiency : 19.73%
>> Effective DMA Bandwidth      : 342.96 MB/s
=====
>> ACCURACY CHECK               : [PASSED] NPU outputs perfectly match CPU!
=====
[4/4] Exporting hardware result to output_C512x512_NPU.bin... Done.
```

Kết quả 512x512

```
[1/4] Loading INT8 data from binary files... Done. Loaded 13107200 bytes successfully.
[2/4] Running CPU Baseline Matrix Multiplication (33.5 Billion MACs)...
[CPU Baseline Progress] [=====] 100%
CPU Baseline Done in 3634156.263 ms.

[3/4] Running NPU: 5120x5120 matmul | K_DIM=32 | TILES=40 | SHIFT=10

=====
>> PERFORMANCE & ACCURACY COMPARISON REPORT <<
=====
Matrix Size           : 5120 x 1280 x 5120 (INT8)
1. CPU Execution Time   : 3634156.263 ms
2. Pure NPU Hardware Time : 11596.750 ms (DMA TX + RX Wait)
3. Total NPU System Time : 84254.965 ms (Includes Linux overhead)
=====
>> SPEEDUP (Pure Hardware) : 313.38x FASTER than CPU
>> SPEEDUP (Total System)  : 43.13x FASTER than CPU
=====
>> SYSTEM LEVEL TIMING PROFILING REPORT <<
=====
1. Software Prep Time (T_sw_prep) : 71767.394 ms ( 85.8%)
2. DMA Transmit + Core (T_dma_tx)  : 11431.208 ms ( 13.7%)
3. DMA Receive Time (T_dma_rx)     : 165.542 ms ( 0.2%)
4. Software Post Time (T_sw_post)   : 245.649 ms ( 0.3%)
=====
>> Hardware Compute Efficiency : 13.87%
>> Effective DMA Bandwidth      : 347.08 MB/s
=====
>> ACCURACY CHECK               : [PASSED] NPU outputs perfectly match CPU!
=====
[4/4] Exporting hardware result to output_C5120x5120_NPU.bin... Done.
```

Kết quả 5120x1280x5120



Kiểm tra kết quả và hiệu năng với Python

Kết quả kiểm tra Bit-True:

BẢNG VÀNG XÁC THỰC PHẦN CỨNG: BIẾN DỊCH TOÁN HỌC PYTORCH VS ĐẦU RA MẠCH RTL NPU				
Phân Lớp Mục Tiêu	Kích Thước Ma Trận	Số Điểm Lỗi Dữ Liệu	Sai Số Max (LSB)	Trạng Thái RTL
Attention_Q_PROJ	512x512 x 512x512	0 / 262,144	0 LSB	✓ MATCH 100%
FFN Upsample FC1	5120x1280 x 1280x5120	0 / 26,214,400	0 LSB	✓ MATCH 100%

Kết quả đánh giá giải lượng tử kết quả GEMM:

Các Chỉ Số Đo Lường (C = A x B)	Output_Q_PROJ (512)	Output_FC1 (5120)	Chuẩn LLM Lý Thuyết
Độ tương đồng Cosine (Hướng)	0.993575	0.998120	> 0.990000
Tỷ số Tín hiệu/Nhiều (SQNR)	18.88 dB	22.69 dB	> 15.00 dB
Sai số tuyệt đối TB (MAE Requant)	0.036134 FP32	0.035945 FP32	Nhỏ nhất có thể
Sai số bình phương gốc (RMSE)	0.043156 FP32	0.097420 FP32	Thấp (< 1.0)
Dải giá trị INT8 thu được trên NPU	[-19, 19]	[-53, 110]	Trong [-128, 127]

05

Kết luận





Kết luận

- Toàn bộ thiết kế được mô tả ở mức RTL bằng Verilog, kiểm chứng chức năng thông qua mô phỏng và triển khai thực tế trên FPGA KV260 để đánh giá tài nguyên sử dụng, timing và công suất tiêu thụ.
- Các quy mô ma trận từ trung bình đến lớn đã minh chứng cho ưu thế vượt trội hoàn toàn của phân vùng logic lập trình (PL) so với bộ vi xử lý trung tâm (PS)
- Kết quả thực nghiệm với các bài toán GEMM thực tế cho thấy hệ thống hoạt động cực kỳ ổn định với bài toán nhân ma trận kích thước 512×512 và $5120 \times 1280 \times 5120$.
- Hướng phát triển: đưa toàn bộ quá trình tiling và điều phối dữ liệu hiện đang được thực hiện trên PS xuống phần cứng PL.



Bảng phân công công việc

Thành viên	Công việc	Đóng góp (%)
Nguyễn Đặng Phương Duy	- Phân tích model Gwen2 - Thiết kế ping-pong buffer - Thiết kế test bench - Thiết kế code C và python - Nạp kiểm thử trên FPGA - Thiết kế slide báo cáo	50%
Vũ Đại Dương	- Thiết kế systolic_array - Thiết kế FSM cho systolic array - Thiết kế skew network - Thiết kế post_accumulator và int32_to_int8_rescale - Viết báo cáo	50%



Cảm ơn cô và các bạn đã lắng nghe